Programação Aplicada em .Net

Aula 4 - Programação Assíncrona

Prof. Me. Lucas Teodoro dos Santos

UNIP

28 de Setembro de 2026

Roteiro da Aula

1. Programação Síncrona
2. Programação Assíncrona
3. Programação Paralela
4. Preparando o café da manhã de forma Síncrona
5. Preparando o café da manhã de forma Assíncrona
6. Task, async e await
7. Aplicações Práticas da Programação Assíncrona

UNIP
UNIVERSIDADE PAULISTA

Modelos de Execução

Em uma aplicação, diferentes tarefas precisam ser executadas: realizar cálculos, ler arquivos, consultar um banco de dados ou acessar uma API.

A forma como essas tarefas são organizadas influencia o comportamento da aplicação durante sua execução.

Nesta aula, vamos trabalhar mais profundamente com a **programação assíncrona**. Antes disso, vamos conhecer os tipos mais comuns de programação relacionados à execução de tarefas:

- **Programação Síncrona;**
- **Programação Assíncrona;**
- **Programação Paralela.**

UNIP
UNIVERSIDADE PAULISTA

Programação Síncrona

Na programação **síncrona**, as tarefas são executadas em sequência. Uma tarefa precisa ser concluída antes que a próxima possa iniciar.

Se uma operação demora, como consultar o banco de dados ou ler um arquivo, a aplicação aguarda seu resultado para continuar o fluxo.

Ideia principal

Executar uma tarefa, aguardar seu término e somente depois continuar.

UNIP
UNIVERSIDADE PAULISTA

Programação Síncrona



Programação Assíncrona

Na programação **assíncrona**, a aplicação pode iniciar uma operação demorada sem permanecer bloqueada durante toda a espera.

Enquanto aguarda uma resposta do banco de dados, de uma API ou de um arquivo, a aplicação pode continuar disponível para outras tarefas.

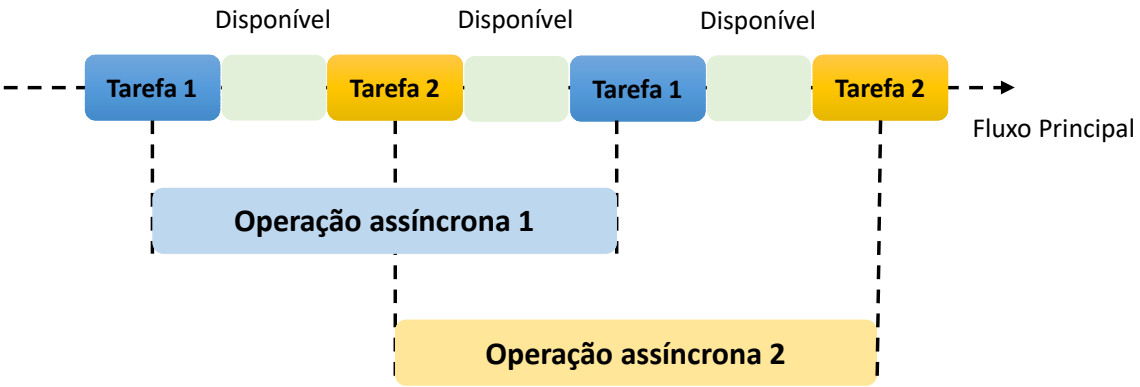
Quando o resultado estiver pronto, o fluxo é retomado no ponto em que ele é necessário.

Ideia principal

Não bloquear a aplicação enquanto uma operação aguarda uma resposta.

UNIP
UNIVERSIDADE PAULISTA

Programação Assíncrona



Programação Paralela

Na programação **paralela**, duas ou mais tarefas são executadas ao mesmo tempo, normalmente utilizando diferentes núcleos do processador.

Esse modelo é útil quando existe uma grande quantidade de processamento que pode ser dividida em tarefas independentes.

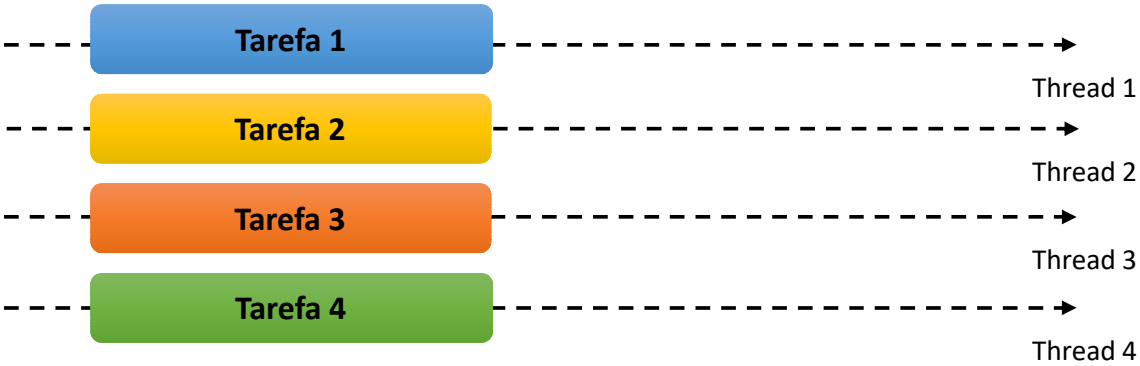
Por exemplo, uma aplicação pode processar várias imagens ou realizar cálculos extensos simultaneamente.

Ideia principal

Executar mais de uma tarefa simultaneamente para reduzir o tempo total de processamento.

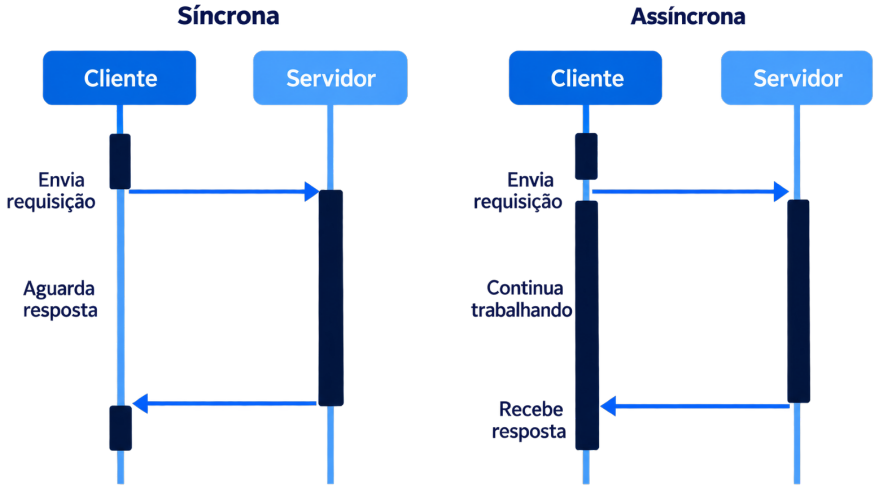
UNIP
UNIVERSIDADE PAULISTA

Programação Paralela



UNIP
UNIVERSIDADE PAULISTA

Síncrona X Assíncrona



Funcionamento da Programação Assíncrona?

Em algumas atividades, iniciamos uma operação e precisamos **esperar** até que ela termine. Durante essa espera, podemos cuidar de outra atividade.

Por exemplo, depois de começar a passar o café, não precisamos ficar observando até que ele esteja pronto. Enquanto esperamos, podemos colocar o pão na torradeira.

Na programação assíncrona, organizamos o código para iniciar uma operação, aproveitar seu tempo de espera e continuar quando seu resultado for necessário.

No próximo exemplo

Vamos comparar o tempo necessário para passar o café e tostar o pão de forma síncrona e assíncrona.

UNIP
UNIVERSIDADE PAULISTA

Exemplo de uma Tarefa do Dia a Dia

Imagine que precisamos realizar duas atividades para preparar o café da manhã:

- Passar o café, levando aproximadamente 10 segundos;
- Tostar o pão, levando aproximadamente 5 segundos.

Ao final, somente podemos servir o café da manhã quando **as duas atividades estiverem concluídas**.

Vamos observar como a organização dessas tarefas altera o tempo total de execução.

UNIP
UNIVERSIDADE PAULISTA

Preparando o café da manhã de forma síncrona

Na execução síncrona, chamamos uma atividade e aguardamos sua conclusão antes de iniciar a próxima.

1. Começamos a passar o café;
2. Após 10 segundos, o café fica pronto;
3. Começamos a tostar o pão;
4. Após mais 5 segundos, o pão fica pronto.

Tempo total aproximado: 15 segundos.

UNIP
UNIVERSIDADE PAULISTA

Café da Manhã: Execução Síncrona

```
1           static void Main(string[] args)
2           {
3               // Inicia a medicao do tempo total de execucao.
4               Stopwatch relogio = Stopwatch.StartNew();
5
6               PassarCafe(); // Passa o cafe e aguarda o metodo terminar.
7               TostarPao(); // So comeca a tostar o pao depois que o cafe esta
                    pronto.
8
9               relogio.Stop(); // Encerra a medicao apos as duas atividades.
10
11              Console.WriteLine("Cafe e pao prontos. Pode servir!");
12
13              Console.WriteLine(
14              $"Tempo total: {relogio.Elapsed.TotalSeconds:F1} segundos");
15          }
```

Café da Manhã: Execução Síncrona - Métodos Síncronos

```
1           static void PassarCafe()
2           {
3               for (int i = 0; i < 10; i++)
4               {
5                   Console.WriteLine($"Passando o cafe {i + 1}");
6                   Thread.Sleep(1000); // Bloqueia o programa por um segundo
7               }
8
9               Console.WriteLine("Cafe pronto");
10          }
11
12          static void TostarPao()
13          {
14              for (int i = 0; i < 5; i++)
15              {
16                  Console.WriteLine($"Tostando o pao {i + 1}");
17                  Thread.Sleep(1000); // Bloqueia o programa por um segundo
18              }
19
20              Console.WriteLine("Pao tostado");
21          }
```

Podemos Aproveitar o Tempo de Espera?

Enquanto esperamos o café passar, podemos começar a tostar o pão.

As duas atividades continuam tendo suas próprias durações:

- Café: aproximadamente 10 segundos;
- Pão: aproximadamente 5 segundos.

Se as esperas ocorrerem no mesmo intervalo, o tempo total ficará próximo de **10 segundos**.

Para representar essa situação em C#, vamos conhecer `Task`, `async` e `await`.

UNIP
UNIVERSIDADE PAULISTA

Task

`Task` é um tipo do .NET que representa uma **operação assíncrona**. Por meio desse objeto, podemos acompanhar sua conclusão e observar se ela terminou com sucesso ou com erro.

O método que inicia a operação pode retornar uma `Task` antes de todo o trabalho ter sido concluído. Assim, quem chamou o método recebe uma representação da operação em andamento.

- `Task`: Operação que não produz um valor de retorno;
- `Task<T>`: Operação que produz um resultado do tipo `T`.

Observação: `Task` é um tipo do .NET, não uma palavra-chave da linguagem C#.

async

`async` é uma palavra-chave usada na declaração de um método para permitir o uso de `await` em seu interior.

Quando o método encontra um `await` cuja operação ainda não terminou, sua execução pode ser suspensa. O método retorna ao chamador e continua posteriormente, quando a operação aguardada for concluída.

Normalmente, um método assíncrono retorna `Task` ou `Task<T>`, permitindo que o chamador acompanhe sua conclusão.

Atenção

A palavra `async`, sozinha, não inicia uma nova thread nem torna o código automaticamente paralelo.

UNIP
UNIVERSIDADE PAULISTA

await

`await` é uma palavra-chave usada para aguardar uma operação assíncrona. As instruções seguintes no **mesmo método** só são executadas após a conclusão da operação aguardada.

Se a operação ainda estiver em andamento, o método pode ser suspenso sem manter uma thread bloqueada durante uma espera assíncrona. Quando a operação termina, o método retoma a execução após o `await`.

Se a operação produzir um resultado, `await` permite obtê-lo. Se ocorrer um erro, a exceção é propagada nesse ponto.

Ideia principal

`await` pausa a continuação do método; isso não significa bloquear uma thread durante toda a espera.

Aplicando `Task`, `async` e `await` no exemplo de café da manhã

Queremos iniciar o preparo do café e, durante sua espera, começar a tostar o pão. Para isso, vamos transformar as duas operações em métodos assíncronos.

- `async`: será usado na declaração dos métodos `PassarCafeAsync()` e `TostarPaoAsync()`;
- `Task`: representará o preparo do café e o preparo do pão enquanto essas operações não terminam;
- `await`: será usado dentro dos métodos para aguardar cada etapa e, no `Main`, para esperar as duas operações terminarem.

O café da manhã só será servido depois que **o café e o pão estiverem prontos**.

UNIP
UNIVERSIDADE PAULISTA

Café da Manhã - Métodos Assíncronos (1/2)

```
1           // O metodo e assincrono e retorna uma Task que
2           // representa a operacao de passar o cafe.
3           static async Task PassarCafeAsync()
4           {
5               // Executa dez etapas de um segundo cada.
6               for (int i = 0; i < 10; i++)
7               {
8                   // Mostra a etapa atual no console.
9                   Console.WriteLine($"Passando o cafe {i + 1}");
10
11                  // Aguarda um segundo sem bloquear a thread.
12                  await Task.Delay(1000);
13              }
14
15              // Executado depois que as dez etapas terminam.
16              Console.WriteLine("Cafe pronto");
17          }
```

Café da Manhã - Métodos Assíncronos (2/2)

```
1           // O metodo e assincrono e retorna uma Task que
2           // representa a operacao de tostar o pao.
3           static async Task TostarPaoAsync()
4           {
5               // Executa cinco etapas de um segundo cada.
6               for (int i = 0; i < 5; i++)
7               {
8                   // Mostra a etapa atual no console.
9                   Console.WriteLine($"Tostando o pao {i + 1}");
10
11                  // Aguarda um segundo sem bloquear a thread.
12                  await Task.Delay(1000);
13              }
14
15              // Executado depois que as cinco etapas terminam.
16              Console.WriteLine("Pao tostado");
17          }
```

Café da Manhã: Método `Main` Assíncrono (1/2)

```
1           // Main retorna Task porque agora utiliza await.
2           static async Task Main(string[] args)
3           {
4               Stopwatch relogio = Stopwatch.StartNew();
5
6               // Cada chamada inicia uma operacao e retorna sua Task.
7               Task cafe = PassarCafeAsync();
8               Task pao = TostarPaoAsync();
9
10              // As duas operacoes foram iniciadas antes da espera.
```

Café da Manhã: Método Main Assíncrono (2/2)

```
            // Aguarda o cafe terminar.
            await cafe;

            // Aguarda o pao, caso ainda nao tenha terminado.
            await pao;

            relogio.Stop();

            // So continua quando as duas operacoes terminam.
            Console.WriteLine("Cafe e pao prontos. Pode servir!");
            Console.WriteLine(
            $"Tempo total: {relogio.Elapsed.TotalSeconds:F1} segundos");
        }
```

UNIP
UNIVERSIDADE PAULISTA

Entendendo a Ordem de Execução

1. `PassarCafeAsync()` inicia e retorna uma `Task` ao encontrar a primeira espera;
2. `TostarPaoAsync()` também inicia;
3. As esperas das duas operações se sobrepõem;
4. `await cafe` garante que o café terminou;
5. `await pao` garante que o pão terminou;
6. A mensagem final é exibida.

Tempo total aproximado: 10 segundos.

UNIP
UNIVERSIDADE PAULISTA

Outros Recursos da Programação Assíncrona

Além de `async`, `await` e `Task`, existem outras construções e métodos usados em operações assíncronas:

- `Task.WhenAll()`: permite aguardar a conclusão de várias tarefas;
- `Task.WhenAny()`: permite aguardar até que uma das tarefas termine;
- `await foreach`: percorre dados que chegam por meio de um fluxo assíncrono.

Terminologia

`async`, `await` e `foreach` são palavras-chave do C#. `Task<T>`, `WhenAll()` e `WhenAny()` são recursos do .NET.

Aplicações Práticas da Programação Assíncrona

Em aplicações reais, muitas operações dependem de uma resposta externa e podem levar algum tempo para terminar.

A programação assíncrona é utilizada, por exemplo, para:

- Consultar ou gravar dados em um banco de dados;
- Enviar requisições para APIs;
- Ler ou gravar arquivos;
- Aguardar respostas em uma comunicação pela rede.

No projeto **Agendamento**, temos um exemplo próximo: a listagem de médicos depende de uma consulta ao banco de dados.

A seguir, vamos observar como adaptar essa operação para aguardar a resposta do banco de forma assíncrona.

Listagem de Médicos: Implementação Atual

```
// MedicoService
public List<Medico> Listar()
{
    return _context.Medicos.ToList();
}

// MedicoController
public IActionResult Index()
{
    var medicos = _medicoService.Listar();
    return View(medicos);
}
```

O método `ToList()` executa a consulta. A requisição aguarda a resposta do banco antes de continuar.

Listagem de Médicos: Implementação Assíncrona

```
1           // MedicoService
2           public async Task<List<Medico>> ListarAsync()
3           {
4               return await _context.Medicos.ToListAsync();
5           }
6
7           // MedicoController
8           public async Task<IActionResult> Index()
9           {
10              var medicos = await _medicoService.ListarAsync();
11              return View(medicos);
12          }
```

Enquanto aguarda a consulta, a thread não precisa permanecer bloqueada. Quando a lista chega, a execução continua e a controller a envia para a view.

Resumo Geral da Aula

- Apresentação dos modelos de programação síncrona, assíncrona e paralela;
- Compreensão das diferenças entre a execução síncrona e a assíncrona;
- Apresentação de `async`, `await` e `Task`;
- Aplicação prática no projeto **Agendamento**.

UNIP
UNIVERSIDADE PAULISTA

Exercício de Fixação: Preparando uma Viagem

Crie um novo projeto Console em C# que simule duas atividades:

- Carregar o celular: 8 segundos;
- Arrumar a mala: 5 segundos.

1. Implemente primeiro a versão síncrona, utilizando `Thread.Sleep();`
2. Meça e exiba o tempo total com `Stopwatch`;
3. Crie uma versão assíncrona dos dois métodos, utilizando `async`, `Task` e `await Task.Delay();`
4. Inicie as duas operações antes de aguardá-las;
5. Exiba "Pronto para viajar!" somente quando ambas estiverem concluídas.

UNIP
UNIVERSIDADE PAULISTA

Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA